ISO/IEC JTC1 SC22 WG21 P1654R0
Roger Orr <rogero@howzatt.demon.co.uk>
Target audience: All of WG21
2019-06-13

ABI breakage - summary of initial comments

Abstract

The Direction Group has been asked to look at ABI (Application Binary Interface) breakage. Note that this is different from, but related to, the API (Application Programming Interface).

The first step was to solicit input from EWG/LEWG to try and capture the variety of issues covered by the single term "ABI breakage".
This document is an informal summary of comments, including many of those made on the two evolution reflectors.

Thank you to all who have provided input; errors and omissions remain, as usual, the fault of the author.

Table of Contents

1. Original statement of the concern

From Michael Wong's original email to EWG/LEWG:

There is a concern that we need to be clear on when we can make clear ABI breakage. There seems to be several cases on a spectrum of Performance vs Stability.

Cases 1-4:

1. NEVER break ABI : this is the hope and the most stable.
2. Break ABI on a case by case basis e.g. SSO (small string optimisation)
3. Break ABI at key declared boundary releases, e.g. allow break every 12 years, or every 4 releases
4. Break ABI at will, e.g. every release, most Performance

Case 1: NEVER Break ABI leads to slower and slower performance but the best stability
Case 2: we have done that before, but it is unpredictable to users
Case 3: We have never tried this before, the question is what is the appropriate time frame between

breakage
Case 4: this is the fastest moving, most performance, and the least stability.

2. Meta Comments

What is meant by "the C++ ABI" ?

There is no *ABI* specified in the C++ standard; implementation details are left to each implementor. However, each implementation will have made a variety of decisions about the ABI.

Some implementations have a published ABI - for example on x64 many implementations conform to the Itanium specification (which applies to more than just the Intel Itanium chipset.)

Some implementations publish guarantees about stability of their ABI. For example, Red Hat publish an "Application Compatibility GUIDE" ; Appendix A guarantees compatability for three releases for various components, including `libstdc++`.

Such documents and guarantees will not, in general, be under the control of WG21.

An ABI covers two separable concerns:

1. The first, foundational, concern is how C++ types and functions map into the underlying architecture. This includes such things as data structure layout, primitive types, calling convention, polymorphism internal data structures, and register usage.
2. The second concern is the ABI of the standard library. This will depend heavily on specific implementation details of library types and classes.

Changes to the first part generally make it extremely difficult to safely combine binaries with old and new ABI into a single executable.

Changes to the library can sometimes be accommodated in the library ABI by using sufficiently clever programming to provide backwards compatibility with an existing binary library.

However, the degree to which this is possible, in any given case, can vary between implementors depending on their library implementation. It can be hard to know, without specialist knowledge, how feasible this might be in a specific case.

Impact on proposals

One thing someone found particularly noteworthy: in the San Diego meeting with its preponderance of new-attendees and papers from new attendees, we saw a significant uptick on papers that were dismissed because of ABI concerns. While that isn't **proof** of anything, I **suspect** that there are many ideas that experienced committee members are filtering early because we have internalized "that's an ABI break and thus a non-starter."

They didn't have anything on my list that individually felt (even to them) like "we should break ABI for this" - the most impactful bit would probably be improvements to hashing. But they **suspect** that if we plan for it with enough lead time we'll come up with a lot of quality-of-life and minor performance improvements that add up to a lot.

Or, if we are just going to let "that's an ABI break" be an automatic veto, we should probably update our published priorities. They don't think "ABI stability" is listed anywhere as a "you can rely on this"

feature for C++.

3. Past ABI breaks

The std::string class was changed for C++11, in response to the addition of threads, to make more of its operations safely concurrently executable (which invalidated copy-on-write implementations.) This included changing `data()` to require NUL termination.

Additonally, the change was designed to support the 'small string optimisation', which improves performance for strings short enough to take advantage of it.

Implementing this requires an ABI change in general as the size of the class changes, as does the layout and meaning of its members.

gcc provided a dual ABI to support both pre- and post- C++11 code, but it is still the case that by default many installations of gcc still use the pre C++11 implementation.

When `::operator new()` started throwing `std::bad_alloc`, two binary incompatibilities were introduced:

1. Existing binaries that were not designed to have exceptions propagating through them suddenly had exceptions propagating through them
2. Existing binaries that checked the return value of `new` no longer handled out of memory conditions

Compilers and runtimes incorporated some devious tricks to mitigate the potentially harmful effects of this necessary change.

The changes to the definition of triviality could potentially have changed calling conventions, but to avoid that the Itanium ABI uses the C++98 definition of POD, not the current definition of trivial and standard layout, because that's evolved over time.

Adding the `std::system_error` base class to `std::ios_failure` for C++11 was a particularly nasty one for one implementor.

> (The reason that one's so troublesome is that changing the type of exception thrown by a library doesn't produce any linkage changes. You can still link to it as before, but suddenly an exception that used to get caught now passes straight through your catch handlers. When a function's return type or parameter type changes, that can be turned into a linker error, so the user knows to recompile. Changes to the type of an exception thrown by the standard library (where the throw site is not in a header, so the precise type thrown is out of your control) is a silent change in runtime behaviour, and only on the exceptional path.
>
> One implementor remarked: "Buy me a beer some time and I'll tell you the story of Schrödinger's Catch, which allows a single catch handler to work for two distinct types of `std::ios_failure`."

In C++11 `std::char_traits` changed the parameter types of its members from `const char_type&` to passing `char_type` by value (It is believeed this is still not actually implemented in libstdc++).

I think the LWG issues list records quite a few breaks between C++98 and C++03 that probably wouldn't be acceptable today, but back then almost nobody actually implemented the full standard anyway, so making breaking changes was just part of finishing the implementation!

P0482 (char8_t) changed the return type of the `u8string` and `generic_u8string` member functions of `std::filesystem::path` for C++20.

4. Implementation related

Whether move constructors should affect whether a type is passed in a register or on the stack

Whether empty class types as function arguments take up a slot in the argument list or not.

5. Changes with 'deprecation'

The removal of `uncaught_exception` wasn't really an ABI break due to zombie names.

Same for get_unexpected/set_unexpected, etc.

6. Changes rejected because of ABI breaks

A selected list of papers that have been to LEWG or LEWG-I and rejected (sometimes without further discussion) because their perceived value **individually** didn't measure up to the perceived cost of an ABI break:

- LEWG1053 (Unify algorithms with operator and function object variants)
- D.7 - remove uncaught_exception?
- system_error should return string_view not std::string
- heterogenous lookup could be smoother
- hashing salt / std::hash optimization (which means standard unordered containers are forever vulnerable to hash flooding)
- push_back returns T&
- int128 / uint128 can't be added (because maxint_t is part of the ABI)
- mark bitset trivial?
- P1196 (Value-based std::error_category comparison)
- P1197 (A non-allocating overload of error_category::message())
- P1198 (Adding error_category::failed())
- P1249 (Allow initializer_list to be of non-const T)
- LWG3211 (std::tuple<> should be trivially constructible)

ABI was the reason why we didn't make destructors implicitly virtual in polymorphic classes. "If we can take an ABI break we can fix that."
Note: the ABI was not the sole reason; and the impact of this change would be massive at this stage in the life of the language.

Adding new virtual functions to `std::num_get` and `std::num_put` was proposed for short float, but it is believed has now been dropped from the proposal.

7. Changes reverted because of ABI breaks

The addition of `std::default_order` to associative containers was reverted because it was an ABI break.

The change from `lock_guard<T>` to `lock_guard` was reverted because it was an ABI break.

8. Future ABI breaks

When we tried to add monadic optionals, we were concerned that we cannot pass overload sets to callables. This (passing overload sets to callables) would require a future planned ABI breakage.

(Not an ABI break taken, but one that should have been (or should be) taken)

make `std::unique_ptr<T>` be passed as efficiently as `T*`. Currently there is a significant performance and optimization hit from using `std::unique_ptr<T>` due to the ABI & calling convention required.

> Which had the following reply: This is slightly different from, say, `list::size` and CoW string; there was no change in the specification that would've caused or prevented such a passing convention. There's fairly little we could've done in the standard to impact this.

Numerous aspects of `std::unordered_map`'s API and ABI force seriously suboptimal implementation strategies. (See "SwissTable" talks.)

Same for `std::map`. (For example btree-based sorted containers.)

The most frustrating for one person is `std::vector`, which cannot support small-size-optimization due to stability of pointers & iterators across move.

We changed the return type of `*::emplace_back` from `void` to return a reference to the new element. We *didn't* do the same for `push_back` because that would have broken ABI. If we could break ABI we could make them consistent and remove one reason to (ab)use `emplace_back`.

> *Further discussion*
>
> Is that really a problematic ABI break for some compilers? In gcc-land we might stick an abi_tag on it so the new version gets a different mangling, but I believe the ABI is not broken. Unless of course you start introspecting and use `decltype(c.push_back(e))`, but that's indirect and seems acceptable.
>
> *In reply:*
>
> Without the abi-tag the old and new versions of the function have the same mangled name.
>
> One translation unit instantiates the old definition, and in that TU nothing uses the return value (because it's void).
>
> Another translation unit instantiates the new definition, and the caller of the function uses the non-void return values.
>
> You have two instantiations, with the same symbol name. The linker picks one. Because it's a Thursday the linker picks the old definition of the symbol, which doesn't actually return anything. The new TU calls the old symbol, and there is junk on the stack where it expects to find a return value.
>
> *Further reply:*
>
> Thanks. Sorry, my message was not clear enough. I know all that. My point was that some annotation like abi-tag easily avoids this issue. And you only need a very basic version of abi-

tag for that, which should be easy to implement for any compiler that cares about binary compatibility. So I don't think we should refrain from making such changes for ABI reasons.

Since this is a member function, its exact signature is not mandated by the standard, so an implementation could also add an extra argument with a default value, as allowed by [member.functions], to give it a different mangling. But a vendor-specific annotation is more convenient.

Library Fundamentals defines `std::packaged_task` and `std::promise` with polymorphic allocator members, which adds a pointer member to the class. That was originally proposed as a change to the standard types when LFTS was enabled, which would have been an ABI break. Instead the types in LFTS are distinct types in a distinct namespace.

LWG2503 (multiline option should be added to syntax_option_type) is an ABI break.

9. Additional resources

See these links for other discussions on binary compatability issues:

- https://community.kde.org/Policies/Binary_Compatibility_Issues_With_C++
- https://community.kde.org/Policies/Binary_Compatibility_Examples
- https://wiki.qt.io/Qt-Version-Compatibility Qt pledged ABI compatibility for major releases (almost 10 years)

There's some tooling around managing the ABI:

- https://abi-laboratory.pro/
- a href="https://sourceware.org/libabigail/">https://sourceware.org/libabigail/
- Proposal: llvm-tapi, adding YAML/stub generation for ELF linking support, which morphed into the new llvm-elfabi tool

(Thanks to Morris Hafner for providing these links)